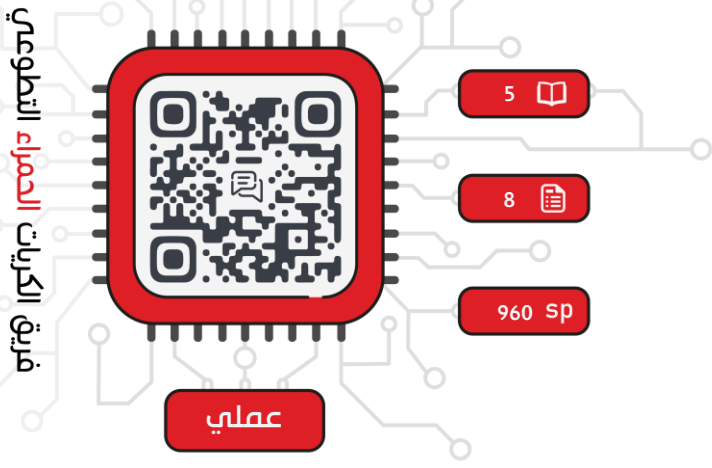


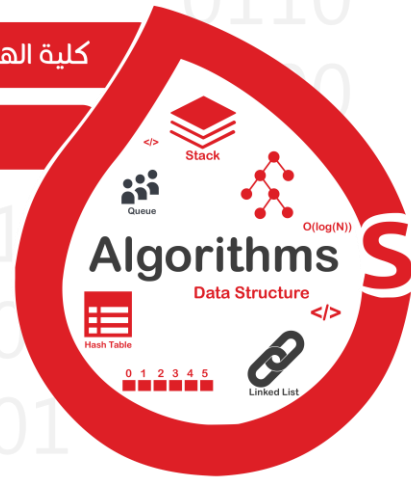


كلية الهندسة المعلوماتية

السنة الثانية

Back track

عملي مشترك



RB Informatcs;

9/12/2025

خوارزميات ا

سنحتاج في هذا البحث لمفهوم العودية+ brute force (تجربة جميع الاحتمالات الممكنة)

تذكرة ببعض الأمور المهمة في العودية:

■ يتألف التابع العودي من :

- مجموعة من التعليمات

- وتعليمه استدعاء التابع لنفسه

- شرط التوقف ويسمى بال base case حيث لا يستدعي التابع نفسه عند هذه الحالة وتتوقف العودية.

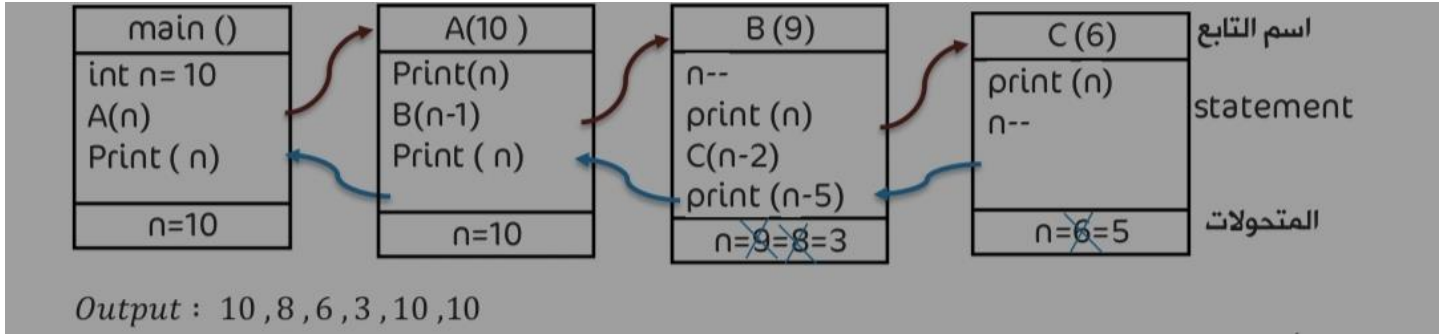
- في كل مرة يستدعي التابع نفسه يتم حجز frame جديد ضمن stack يخزن بها المتحولات المحلية الخاصة بهذا الاستدعاء ورقم سطر الاستدعاء أو عنوان العودة return address
- في حال لم ينفذ شرط التوقف واستمرت الاستدعاءات العودية يرمي stack over flow Exception
- ليس كل تابع عودي يمكن كتابته ككود تكراري، ولكن كل كود تكراري يكتب ككود عودي.

مثال:

لتكن لدينا التتابع الآتية:

```
static void C(int n){
    System.out.println(n);
    n--;
}
static void B(int n){
    n--;
    System.out.println(n);
    C(n-2);
    System.out.println(n-5);
}
static void A(int n){
    System.out.println(n);
    B(n-1);
    System.out.println(n);
}
```

```
public static void main(String[] args) {
    int n=10;
    A(n);
    System.out.println(n);
}
```



نلاحظ أن التنفيذ يبدأ من main حيث يتوقف عند استدعاء التابع A إلى حين استكمال تنفيذه، حيث سيتوقف أيضاً عند استدعاء التابع B ليستدعي التابع C ويتم تنفيذه ثم نستكمل تنفيذ التوابع A, B, main على التوالي.

نلاحظ أيضاً أن لكل تابع frame الخاصة به حيث يخزن متغيراته الخاصة، فنرى مثلاً في التابع B أول قيمة تصل له $n=10$ ثم تصبح 8 ثم يستدعي التابع $c(n-2)$ لكن هذه القيمة خاصة بالتابع C فعند الطباعة بالتعليمة الأخيرة تكون القيمة 3 بالاستناد على قيمة n قبل استدعاء التابع C وهي 8.

مسألة عودية :

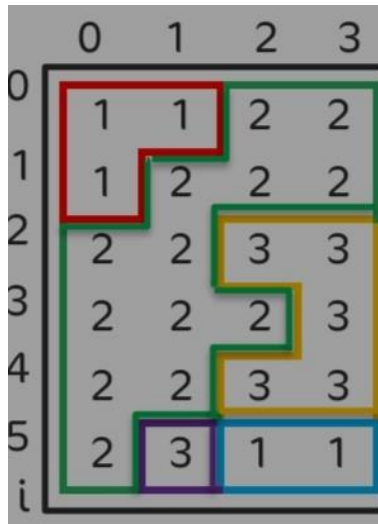
لدينا مصفوفة ثنائية الأبعاد تمثل رقعة مربعات، كل خلية فيها عنصر (كائن) يحتوي على:

color لون الخلية

visited متحول بولياني يمثل هل تمت زيارة هذه الخلية أم لا وهو للمساعدة في إيجاد الحل كي لا يحدث stack over flow من خلال الاستمرار بالدخول إلى نفس الخلايا.

المعطيات: اندكس خلية

المطلوب: إيجاد كل الخلايا المجاورة لها (أعلى - أسفل - يمين - يسار) والتي تحمل نفس اللون، ووضعها جميعاً داخل قائمة component



```

public class Cells {
    boolean isVisit;
    String color;
    public Cells(String color) {
        this.color = color;
        this.isVisit=false;
    }
}

class Function{
    public static void findComponent(Cells[][] grid, int row, int col, String targetColor, List<int []>
    component) {
        // التحقق من حدود المصفوفة
        if (row < 0 || row >= grid.length || col < 0 || col >= grid[0].length) {
            return;
        }
        Cells cell = grid[row][col];
        // التحقق إذا تمت زيارة الخلية أو لونها مختلف
        if (cell.isVisit || !cell.color.equals(targetColor)) {
            return;
        }
        cell.isVisit = true;
        component.add(new int[]{row, col}); // إضافة إحداثيات الخلية إلى المكوّن
        findComponent(grid, row - 1, col, targetColor, component); // أعلى
        findComponent(grid, row + 1, col, targetColor, component); // أسفل
        findComponent(grid, row, col - 1, targetColor, component); // يسار
        findComponent(grid, row, col + 1, targetColor, component); // يمين
    }
}

```

دعونا نتعرّف على المكوّن الثاني من مكوّنات طبختنا (back track):

يشير مصطلح (القوة الغاشمة) Brute Force إلى الأسلوب المتبع في كتابة الخوارزميات والذي لا يتضمّن أي وسيلة لتحسين أداء تلك الخوارزمية، ولكنه يعتمد اعتماداً كاملاً على تجربة جميع الاحتمالات الممكنة إلى حين الوصول إلى الحل المنشود.

back track هو خوارزمية لحلّ المشكلات، يعتمد على بناء الحلّ خطوة بخطوة، وتجربة جميع الاحتمالات الممكنة، حيث تتراجع عن المسارات التي لا تؤدي النتيجة المطلوبة إلى نقطة القرار السابقة لتجربة مسار آخر وهي تعتمد على الاستدعاء الذاتي. (Recursion).

Example: Knapsack Problem:

بفرض لدينا الأغراض التالية والتي لها وزن وقيمة ما:

	Weight	Price
A	20	100
B	10	60
C	40	200
D	15	80
E	5	150

ولنفرض أن شخص ما أراد أخذ كمية من الأغراض بشرط ألا يتجاوز وزنها سعة الحقيبة ولتكن 50 مثلاً و المطلوب تحديد الحد الأقصى للقيمة التي يمكن الحصول عليها مع الحفاظ على الوزن الإجمالي ضمن سعة معينة.

الحل: في هذه الخوارزمية تتم مناقشة جميع حالات المسألة وكل حالة تتألف من عدة خطوات وعند كل غرض نواجه أحد الخيارين وكل خيار ينفذ على شكل استدعاء عودي

الخيار الأول: لا نختار الغرض الحالي ذو الرقم i ينفذ الاستدعاء العودي $bt(i+1, w)$

(حيث ننتقل إلى الغرض الذي يليه من خلال استدعاء التابع من أجل $i+1$ ويبقى الوزن الحالي w نفسه)

الخيار الثاني: نختار الغرض الحالي ذو الرقم i ينفذ الاستدعاء العودي $bt(i+1, w+w[i])$

(في حال اختيار الخيار الثاني نعدل على باراميتر وزن الحقيبة ليضاف إليه وزن الغرض الذي قررنا أخذه , أما بالنسبة للسعر فيضاف سعر هذا الغرض إلى المجموع (sum)

الحل:

```
bt(int i , int w ) {
    if(i==5){
if(sum> maxsum)
    maxsum=sum;
    return; }
    if(weight>=w){
        sum+=price[i];
        bt(i+1,w+w[i]);
        sum-=price[i];
        bt(i+1,w)}
    }
```

شرح الكود:

Weight سعة الحقيبة

$w+w[i]$ وزن الحقيبة الحالي + وزن الغرض الذي تم أخذه

w وزن الحقيبة الحالي

$Price[i]$ سعر الغرض

sum مجموع أسعار الأغراض التي تم اختيار أخذها ضمن الحالة الواحدة ولكل حالة sum خاص بها.

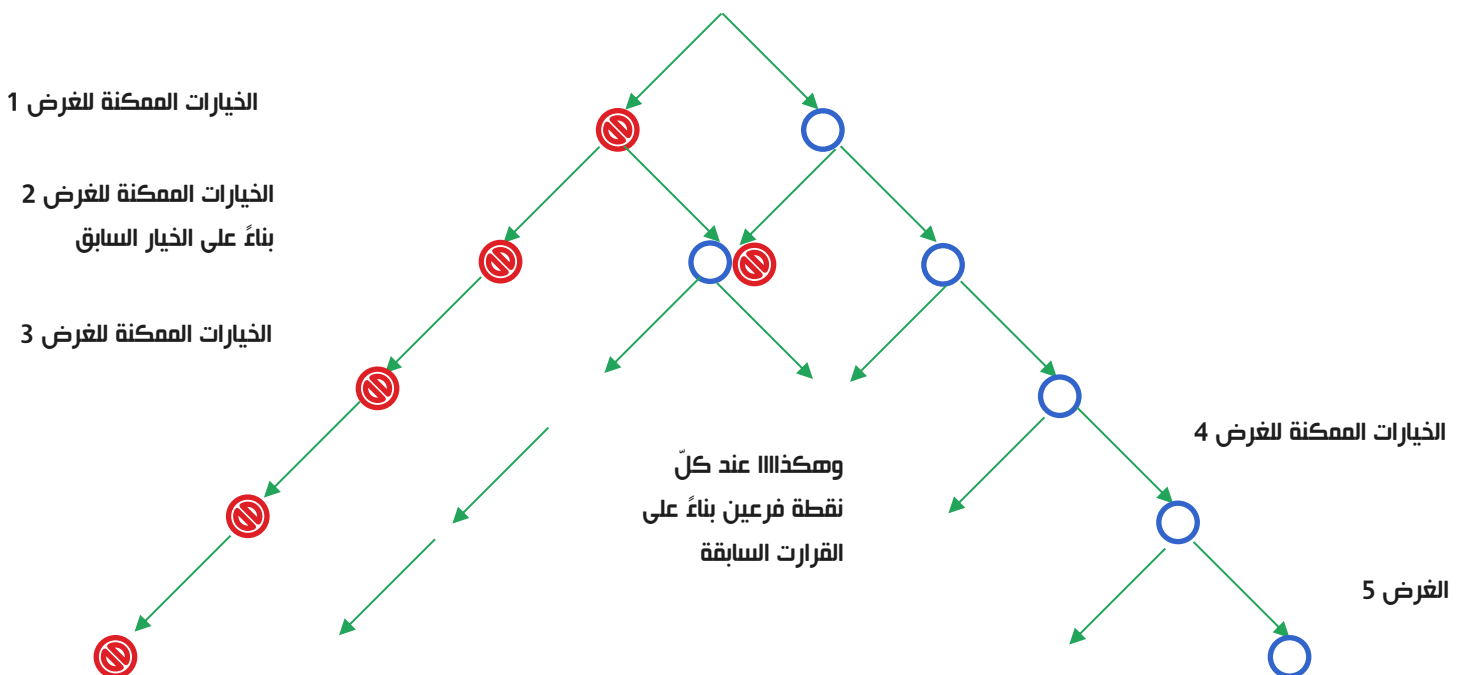
maxsum أفضل حل من بين جميع الحالات وهو الحل الأمثل والجواب النهائي للمسألة

شرح آخر:

أي لدينا شجرة تفرعات لكل غرض هناك قراراتين الأخذ أو الترك حيث ويتج عن كل قرار فرع جديد: الغرض الثاني إما أخذه (إذا كان الوزن أقل من وزن الحقيقة) أو تركه، وذلك عند قرار الأخذ نقوم باستدعاء التابع من أجل الغرض التالي وصولاً إلى آخر غرض ثم نعود لاستكمال تنفيذ كل استدعاء لأخذ خيار الترك لكل عنصر من الأخير للأول ومع كل قرار ترك هناك استدعاء عودي جديد (ما يترتب على هذا الاستدعاء من أخذ أو ترك للأغراض التالية)

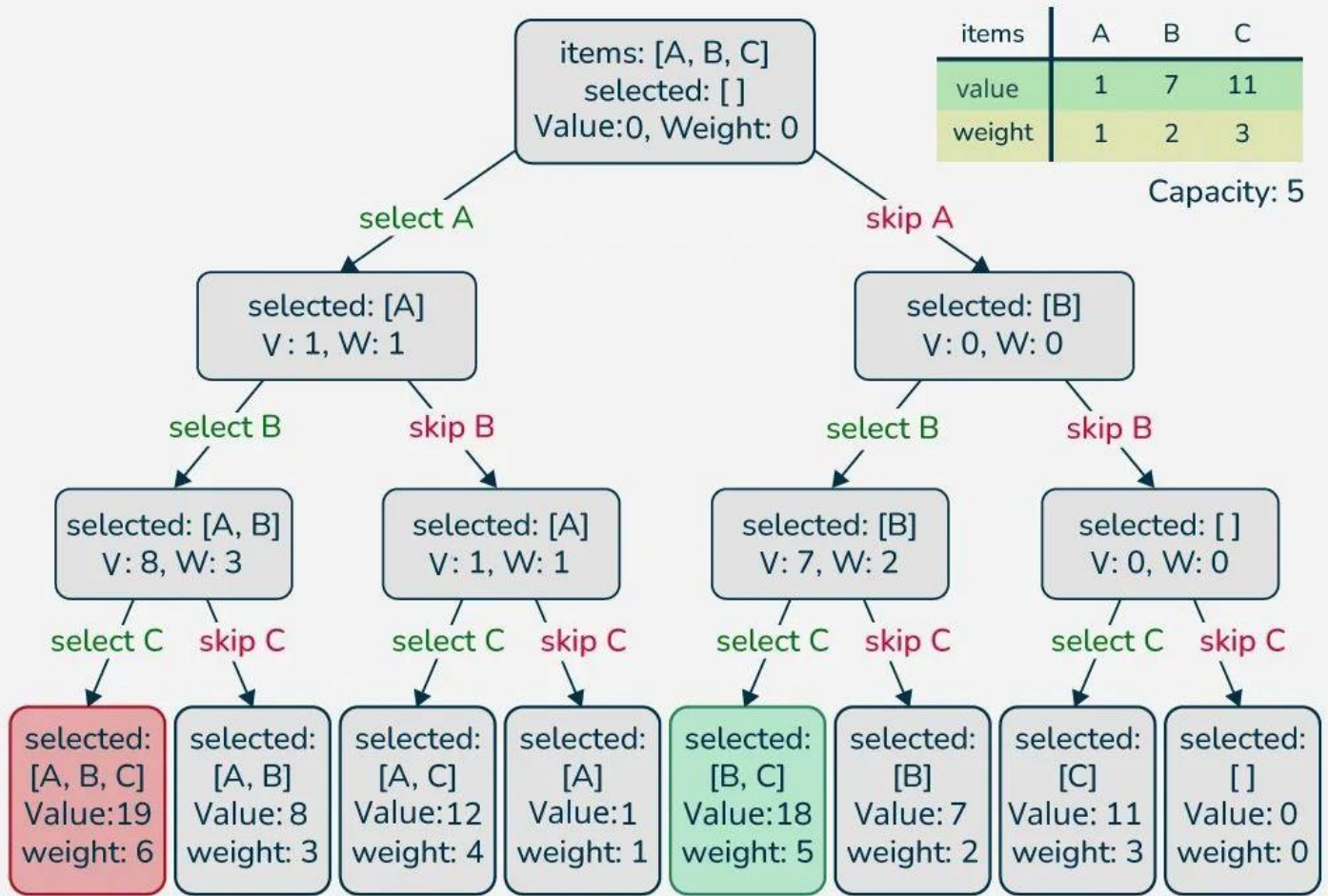
وفي كل مسار من التفرعات سيكون لدينا `sum` سنقوم بتخزين أفضل نتيجة `if(sum > maxsum)`.

طبيب نشوف الرسمة عسانا نفهم : حيث الدائرة الزرقاء قرار الأخذ والحمراء قرار الترك:



يمكن أن لا نتمكّن من أخذ جميع العناصر كما في المسار الأول وذلك حسب حجم الحقيبة وحجم الأشياء الخمسة، لكنّ الرسم هنا للتعبير عن طريقة التفكير بشكل عام عند تطبيق back track.

صورة أخرى لأرقام وسعات مختلفة عن نفس الفكرة :

**مثال 2:** توليد كل التباديل permutations لحروف سلسلة : مثال " ABC " :

نعتبر أن لدينا مواضع (مؤشرات) نثبتها واحداً تلو الآخر من اليسار إلى اليمين.

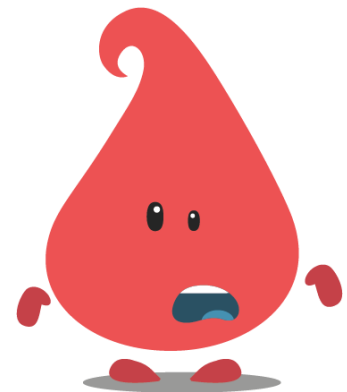
عند خطوة i نجرب وضع كل حرف ممكن من المواضع $l..r$ في الموضع i عن طريق عملية القلب swapبعد كل تبديل نستدعي الدالة بشكل تراجعي على الموضع التالي. $i+1$

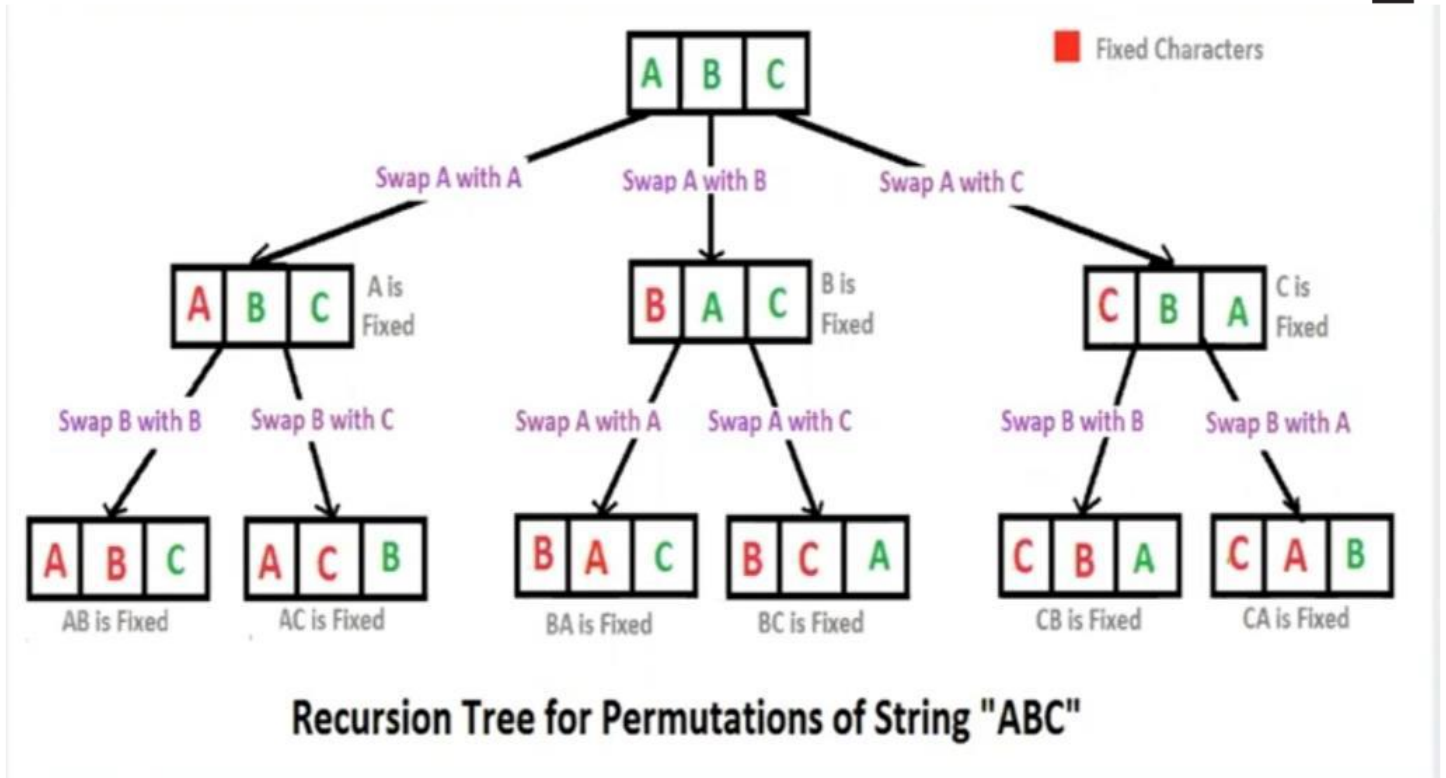
بعد العودة من الاستدعاء نُرجع الحالة إلى ما كانت عليه (أي خطوة التراجع تكون عن طريق عملية القلب swap مجدداً) — هذا هو جوهر التراجع حتى نجرب الخيارات الأخرى.

```

public static void permute(char[] arr, int l, int r) {
    if (l == r) {
        System.out.println(arr);
        return;
    }
    for (int i = l; i <= r; i++) {
        swap(arr, l, i);
        permute(arr, l + 1, r);
        swap(arr, l, i);
    }
}

```

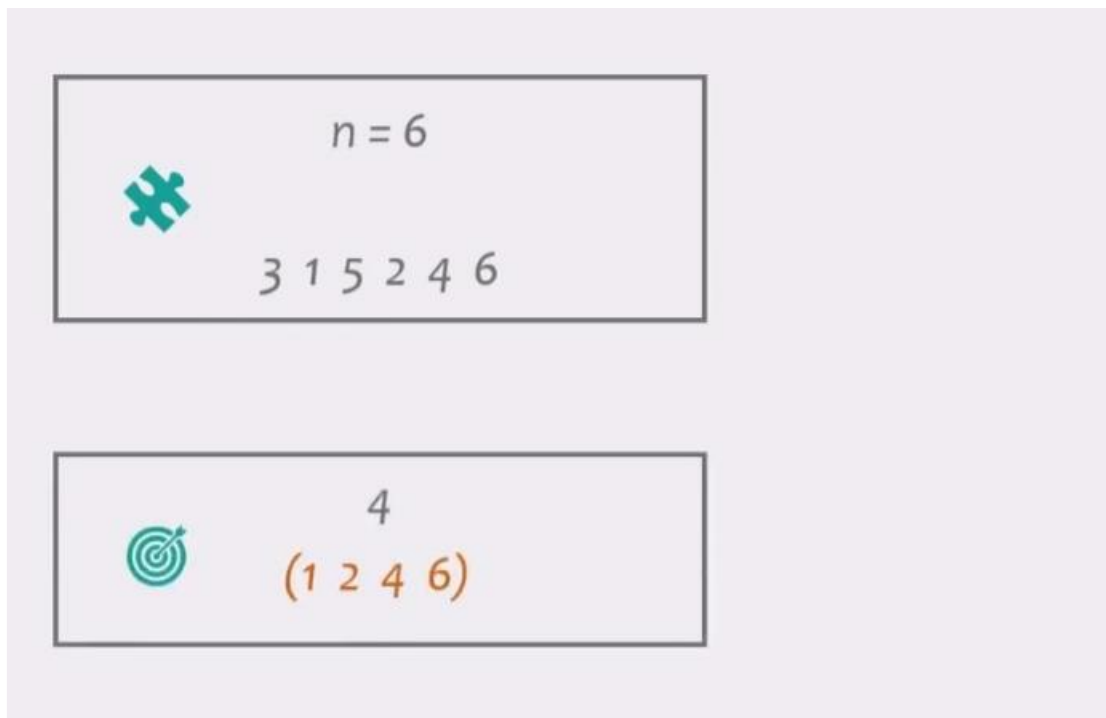




ملاحظة: في حال قمت بتشغيل هذا الكود سيعطي خطأ، بالطبع فأنت لم تقم بتعريف دالة swap حيث لا يوجد دالة جاهزة باللغة تقوم بذلك، وجب التنويه 😊.

مثال 3:

لدينا مصفوفة أعداد صحيحة $a[0 \dots n-1]$ والمطلوب إيجاد طول أطول متتالية تزايدية تحافظ على ترتيب العناصر من اليسار إلى اليمين.



`int a[n];`

```

int lis(int i,int last){// اسم التابع
if(i==n)return; // وصلت لنهاية المصفوفة المعطاة
int ans1=0,ans2=0; // لمعرفة أفضل نتيجة
ans1=lis(i+1,last); // في حال تركنا العنصر الحالي استدعينا التابع من أجل
العنصر التالي
if(a[i]>last)ans2=lis(i+1,a[i])+1; // أخذنا العنصر الحالي وزدنا طول المتتالية
return max(ans1,ans2);
}

```

في كل موضع i من المصفوفة لدينا قراران أساسيان:

- أن نتجاهل العنصر الحالي (leave)
ننتقل إلى الموقع التالي دون إضافة $a[i]$ إلى السلسلة المتزايدة. بالتالي نبحث عن أطول سلسلة متزايدة تبدأ من $i+1$ مع بقاء $last$ كما هو — أي قيمة آخر عنصر قمنا باختياره سابقاً:

$ans1 = lis(i + 1, last);$ // leave

- أن نختار العنصر الحالي (pick)
لكن بشرط أن يكون عنصر المصفوفة الحالي $a[i]$ أكبر من قيمة $last$ أي أكبر من آخر عنصر اخترناه في السلسلة حتى الآن (إذا تحقق الشرط:
نضيف هذا العنصر إلى السلسلة + نزيد الطول بمقدار 1 + ننتقل للموقع $i+1$ مع تحديث $last = a[i]$

$if (a[i] > last)$
 $ans2 = 1 + lis(i + 1, a[i]);$ // pick



المحاضرة صعبة بس أنت وحش